



Федеральное государственное автономное образовательное

учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное
обеспечение

Дисциплина «Вычислительная математика»

Отчёт по лабораторной работе №5

Вариант - 6

Выполнил

Линейский Аким Евгеньевич

P3215

Проверил

Наумова Надежда Александровна

Санкт - Петербург 2026

Содержание

1. Содержание.....	2
2. Цель	3
3. Порядок выполнения работы.....	3
4. Вычислительная часть	4
Таблица значений функции	4
Значений точек по варианту.....	4
Таблица конечных разностей.....	5
5. Формула Ньютона.....	5
6. Формула Гаусса.....	6
7. Многочлен Лагранжа.....	7
Для точки X_1	7
Для точки X_2	7
8. Анализ результатов.....	7
9. Программная часть	8
Интерфейс программы	9
10.Блок схемы численных методов.....	8
11.Исходный программы.....	10
12.Вывод	13

Цель

Решить задачу интерполяции, найти значения функции при заданных значениях аргумента, отличных от узловых точек.

Порядок выполнения работы

1. Вычислительная реализация задачи
 - 1.1. Построить таблице разделенных разностей для заданной таблицы
 - 1.2. Вычислить значения функции для аргумента X_1 , используя интерполяционную формулу Ньютона для не равностоящих узлов.
 - 1.3. Вычислить значения функции для аргумента X_2 , используя первую или вторую интерполяционную формулу Гаусса.
 - 1.4. Вычислить значения функции для аргумента X_1 , X_2 с помощью многочлена Лагранжа.
2. Программная реализация задачи
 - 2.1. Программа
 - 2.1.1. Реализовать ввод исходных данных
 - 2.1.1.1. В виде набора данных (таблицы X , Y) – ввод с клавиатуры
 - 2.1.1.2. В виде сформированных в файле данных
 - 2.1.1.3. На основе выбранной функции, интервала и количества точек на интервале
 - 2.1.2. Сформировать и вывести таблицу разделенных/конечных разностей
 - 2.1.3. Вычислить приближенное значение функции для заданного значения аргумента, введенного с клавиатуры, указанными методами. Сравнить полученные значения
 - 2.1.4. Построить графики заданной функции с отмеченными узлами интерполяции и интерполяционного многочлена Ньютона (разными цветами)
 - 2.1.5. Протестировать программу на различных наборах данных, в том числе и некорректных.

2.1.6. Проанализировать результаты работы программы.

2.2. Реализация схем

2.2.1. Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Стирлинга

2.2.2. Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Бесселя.

Вычислительная часть

Таблица значений функции

X	Y
0.25	1.2557
0.30	2.1764
0.35	3.1218
0.40	4.0482
0.45	5.9875
0.50	6.9195
0.55	7.8359
1.2	1.783590
1.4	1.707039
1.6	1.529441
1.8	1.309281
2.0	1.090909

Табл 1. Таблица значений функции по варианту.

Значений точек по варианту

$$X_1 = 0.512 ; X_2 = 0.372$$

Таблица конечных разностей

$$\Delta y_i = y_{i+1} - y_i, \Delta^k y_i = \Delta^k y_{i+1} - \Delta^k y_i$$

$$h = 0.05$$

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0.25	1.2557	0.9207	0.0247	-0.0437	1.0756	-4.1277	10.1917
0.30	2.1764	0.9454	-0.0190	1.0319	-3.052	6.0640	
0.35	3.1218	0.9264	1.0129	-2.0202	3.0119		
0.40	4.0482	1.9393	-1.0073	0.9917			
0.45	5.9875	0.9320	-0.0156				
0.50	6.9195	0.9164					
0.55	7.8359						

Формула Ньютона

$X_1 = 0.512$ находится в правой половину отрезка. Значит формула интерполирования назад.

$$N_n(x) = y_n + t\Delta y_{n-1} + t * \frac{t(t+1)}{2!} \Delta^2 y_{n-2} + \dots$$

$$+ \frac{t(t+1) \dots (t+n-1)}{n!} \Delta^n y_0$$

$$t = \frac{x - x_n}{h} = \frac{0.512 - 0.55}{0.05} = -0.76$$

$$y_n = 7.8359$$

$$t * \Delta y_{n-1} = (-0.76) * 0.9164 = -0.696464$$

$$\frac{t(t+1)}{2!} * \Delta^2 y_{n-2} = \frac{-0.76 * 0.24}{2} * (-0.0156) = 0.00142272$$

$$\frac{t(t+1)(t+2)}{3!} * \Delta^3 y_{n-3} = \frac{-0.76 * 0.24 * 1.24}{6} * 0.9917 = -0.037379$$

$$\frac{t(t+1)(t+2)(t+3)}{4!} * \Delta^4 y_{n-4} = \frac{-0.76 * 0.24 * 1.24 * 2.24}{24} * 3.0119$$

$$= -0.06357$$

$$\frac{t(t+1)(t+2)(t+3)(t+4)}{5!} * \Delta^5 y_{n-5}$$

$$= \frac{-0.76 * 0.24 * 1.24 * 2.24 * 3.24}{120} * 6.0640 = -0.08294$$

$$\frac{t(t+1)(t+2)(t+3)(t+4)(t+5)}{6!} * \Delta^6 y_{n-6}$$

$$= \frac{-0.76 * 0.24 * 1.24 * 2.24 * 3.24 * 4.24}{720} * 10.1917 = -0.09850$$

$$N_6(X_1) = 6.8585$$

Формула Гаусса

Центральный узел $a = 0.40$; $y_0 = 4.0482$

$$X_2 = 0.372 < a = 0.40$$

Используем вторую интерполяционную формулу Гаусса

$$P_n(x) = y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!} \Delta^2 y_{-1} + \frac{t(t+1)(t-1)}{3!} \Delta^3 y_{-2}$$

$$+ \frac{t(t+1)(t-1)(t+2)}{4!} \Delta^4 y_{-3}$$

$$t = \frac{x - x_0}{h} = \frac{0.372 - 0.40}{0.05} = -0.56$$

$$y_0 = 4.0482$$

$$t * \Delta y_{-1} = (-0.56) * 0.9264 = -0.518784$$

$$\frac{t(t+1)}{2} * \Delta^2 y_{-1} = \frac{(-0.56) * 0.44}{2} * 1.0129 = -0.124789$$

$$\frac{t(t+1)(t-1)}{6} * \Delta^3 y_{-2} = \frac{(-0.56) * 0.44 * (-1.56)}{6} * 1.0319 = 0.066107$$

$$\frac{t(t+1)(t-1)(t+2)}{24} * \Delta^4 y_{-3} = \frac{(-0.56) * 0.44 * (-1.56) * 1.44}{24} * 1.0756$$

$$= 0.24806$$

$$P(X_2) = 3.4955$$

Многочлен Лагранжа

$$L_n(x) = \sum_{i=0}^n y_i * \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

$$L_6(x) = y_0 * \frac{(x - x_1) \dots (x - x_6)}{(x_0 - x_1) \dots (x_0 - x_6)} + \dots + y_6 * \frac{(x - x_0) \dots (x - x_5)}{(x_6 - x_0) \dots (x_6 - x_5)}$$

Для точки X_1

$$L_6(0.512) = 1.2557 * (-0.009667) + 2.1764 * 0.071678 + 3.1218$$

$$* (-0.234504) + 4.0482 * 0.452258 + 5.9875 * (-0.612736)$$

$$+ 6.9195 * 1.266322 + 7.8359 * 0.066648 = 6.8584$$

Для точки X_2

$$L_6(0.372) = 1.2557 * 0.0070 + 2.1764 * (-0.0610) + 3.1218 * 0.3640$$

$$+ 4.0482 * 0.8180 + 5.9875 * (-0.1940) + 6.9195 * 0.0500$$

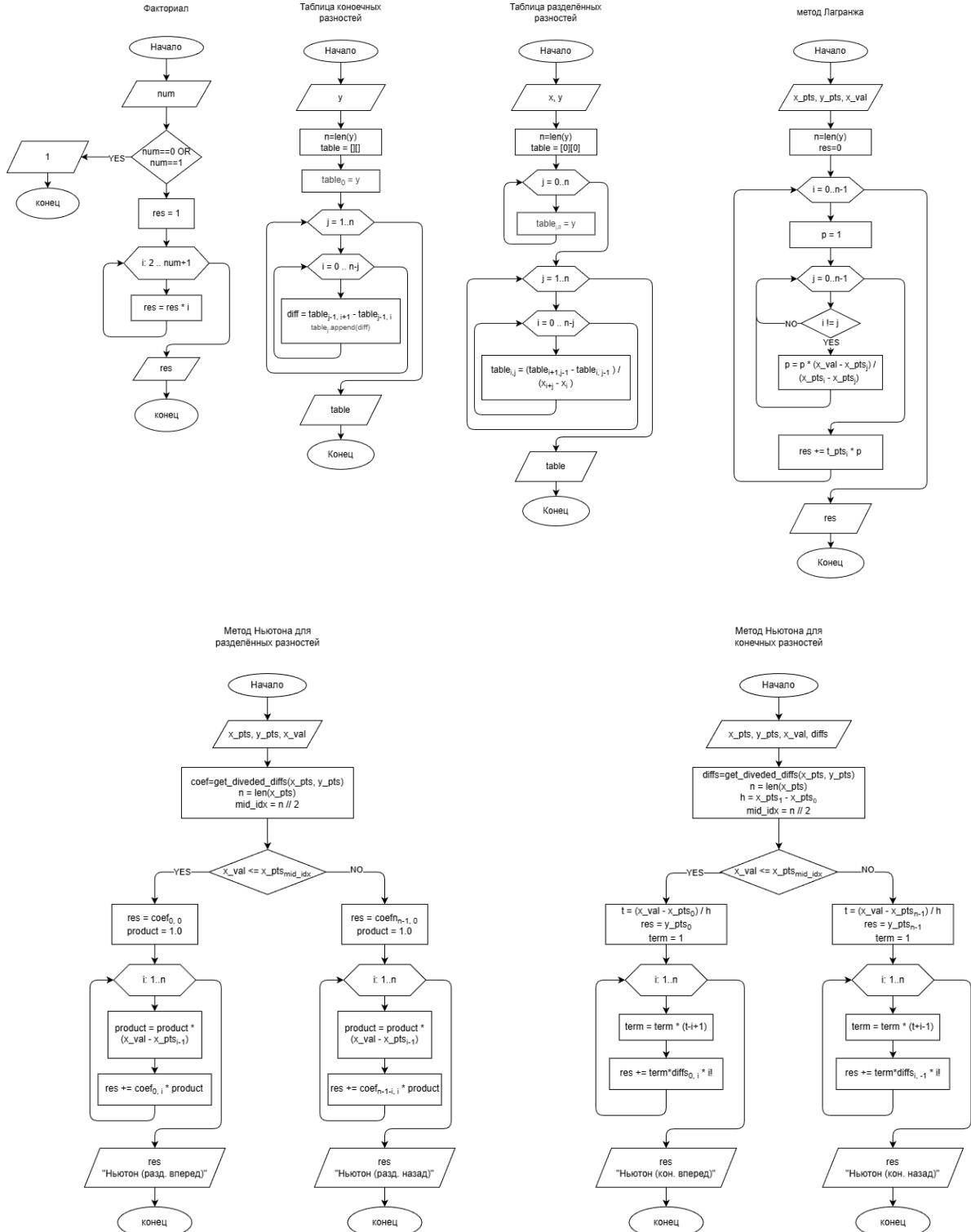
$$+ 7.8359 * (-0.0060) = 3.400$$

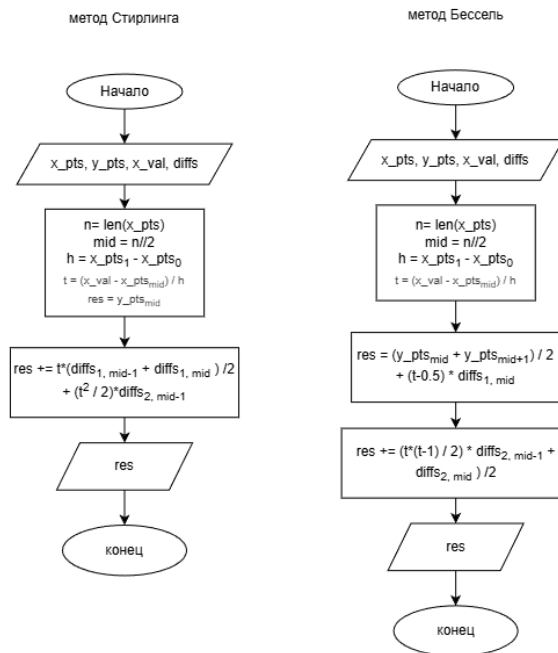
Анализ результатов

Метод	Формула	Точка	Значение
Ньютона	2 формула назад	0.512	6.8585
Лагранжа	Многочлен Лагранжа	0.512	6.8584
Гаусса	2 формула назад	0.372	3.955
Лагранжа	Многочлен Лагранжа	0.372	3.400

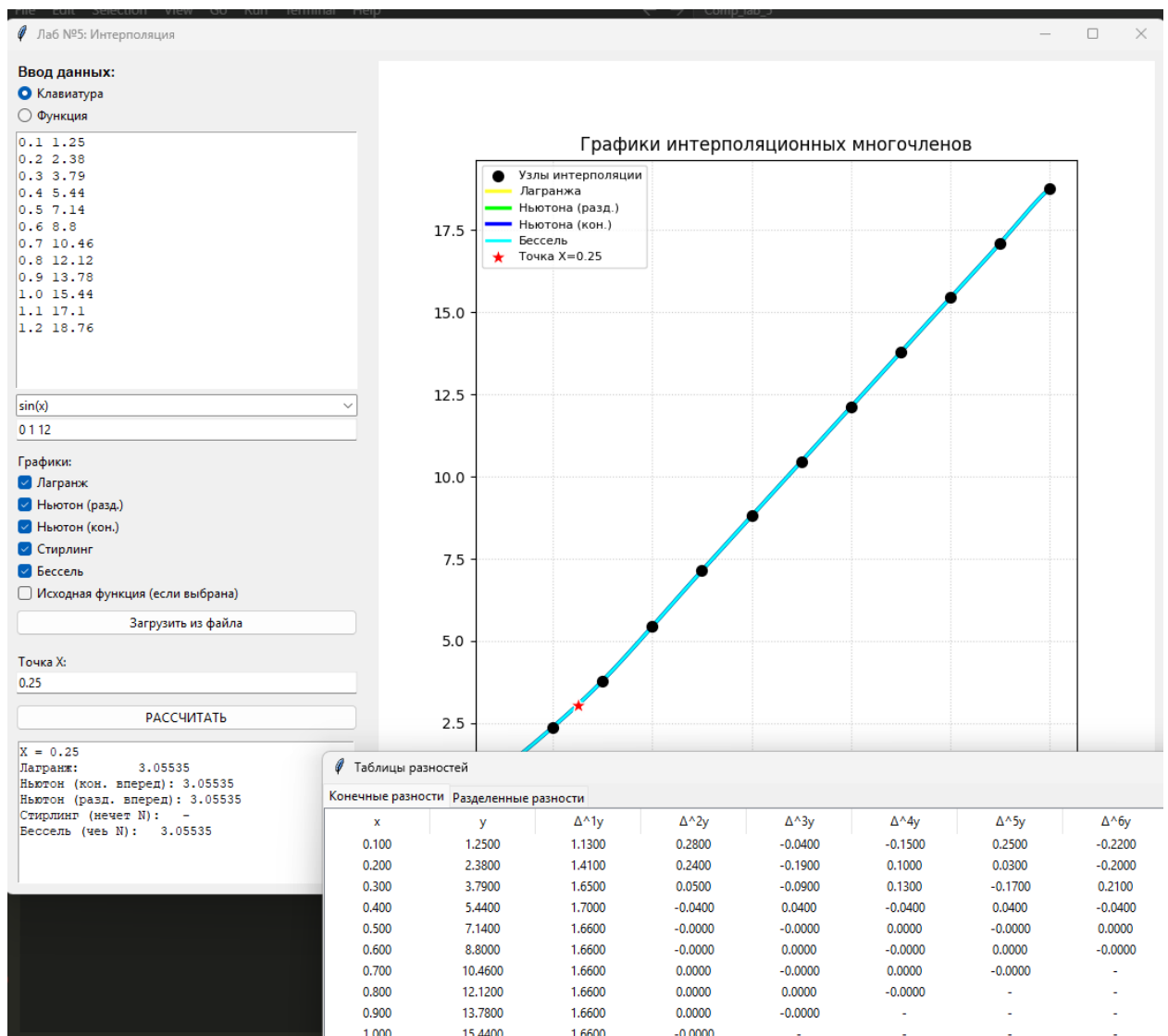
Программная часть

Блок схемы численных методов





Интерфейс программы



Исходный программы

```
# --- МАТЕМАТИЧЕСКИЙ БЛОК ---
class InterpolationMath:
    def factorial(num):
        if num == 0 or num == 1:
            return 1
        res = 1
        for i in range(2, num + 1):
            res *= i
        return res

    def get_finite_diffs(y):
        n = len(y)
        table = [[] for _ in range(n)]
        table[0] = [float(val) for val in y]

        for j in range(1, n):
```

```

        for i in range(n - j):
            diff = table[j - 1][i + 1] - table[j - 1][i]
            table[j].append(diff)
        return [np.array(col) for col in table if col]

    @staticmethod
    def get_divided_diffs(x, y):
        n = len(y)
        table = np.zeros((n, n))
        table[:, 0] = y
        for j in range(1, n):
            for i in range(n - j):
                table[i][j] = (table[i + 1][j - 1] - table[i][j - 1]) / (
                    x[i + j] - x[i]
                )
        return table

    @staticmethod
    def lagrange(x_pts, y_pts, x_val):
        n = len(x_pts)
        res = 0
        for i in range(n):
            p = 1
            for j in range(n):
                if i != j:
                    p *= (x_val - x_pts[j]) / (x_pts[i] - x_pts[j])
            res += y_pts[i] * p
        return res

    @staticmethod
    def newton_divided_auto(x_pts, y_pts, x_val):
        coef = InterpolationMath.get_divided_diffs(x_pts, y_pts)
        n = len(x_pts)
        mid_idx = n // 2

        if x_val <= x_pts[mid_idx]:
            # 1-я формула (вперед)
            res = float(coef[0, 0])
            product = 1.0
            for i in range(1, n):
                product *= x_val - x_pts[i - 1]
                res += float(coef[0, i]) * product
            return res, "Ньютон (разд. вперед)"
        else:
            # 2-я формула (назад)
            res = float(coef[n - 1, 0])
            product = 1.0
            for i in range(1, n):
                product *= x_val - x_pts[n - i]
                res += float(coef[n - 1 - i, i]) * product
            return res, "Ньютон (разд. назад)"

    @staticmethod

```

```

def newton_finite_auto(x_pts, y_pts, x_val):
    n = len(x_pts)
    h = x_pts[1] - x_pts[0]
    diffs = InterpolationMath.get_finite_diffs(y_pts)
    mid_idx = n // 2
    if x_val <= x_pts[mid_idx]:
        # 1-я формула (вперед)
        t = (x_val - x_pts[0]) / h
        res = float(y_pts[0])
        term = 1.0
        for i in range(1, n):
            term *= t - i + 1
            res += (term * float(diffs[i][0])) / factorial(i)
        return res, "Ньютон (кон. вперед)"
    else:
        # 2-я формула (назад)
        t = (x_val - x_pts[n - 1]) / h
        res = float(y_pts[n - 1])
        term = 1.0
        for i in range(1, n):
            term *= t + i - 1
            res += (term * float(diffs[i][-1])) / factorial(i)
        return res, "Ньютон (кон. назад)"

@staticmethod
def stirling(x_pts, y_pts, x_val, diffs):
    n = len(x_pts)
    mid = n // 2
    h = x_pts[1] - x_pts[0]
    t = (x_val - x_pts[mid]) / h
    res = y_pts[mid]
    try:
        res += (
            t * (diffs[1][mid - 1] + diffs[1][mid]) / 2
            + (t**2 / 2) * diffs[2][mid - 1]
        )
    except:
        pass
    return res

@staticmethod
def bessell(x_pts, y_pts, x_val, diffs):
    n = len(x_pts)
    mid = (n - 1) // 2
    h = x_pts[1] - x_pts[0]
    t = (x_val - x_pts[mid]) / h
    res = (y_pts[mid] + y_pts[mid + 1]) / 2 + (t - 0.5) * diffs[1][mid]
    try:
        res += (t * (t - 1) / 2) * (diffs[2][mid - 1] + diffs[2][mid]) / 2
    except:
        pass
    return res

```

Код доступен по ссылке на удаленный репозиторий GitHub:

[Код программы](#)

Вывод

Выполнив данную лабораторную работу №5, я изучил различные способы и типы формул интерполяции функций, реализовал численные методы и графический интерфейс с просмотром графиков аппроксимации на языке Python.